hblibDatastructures User Manual
for hblibDatastructures.sty v20261112T0930

Haluk Bingol

Contents

1. Getting started 1
2. Quick reference 1
3. Colors 2
4. Strings and arrays 2
4.1. \hString: characters and substring 2
4.2. \hArray: the keys 2
4.2.1. Keys in use 3
4.2.2. Node names, labels and reference arrows 3
4.2.3. A loop, step by step 3
5. Two-dimensional arrays 4
5.1. An array of rows 4
6. Linked lists 4
6.1. \hDLList: a doubly linked list 4
6.1.1. After adding a node 5
6.1.2. One node, the empty list, two lists in one picture 5
6.1.3. Node names: labels and arrows of your own 5
6.2. \hSLList: a singly linked list 6
6.2.1. Values of different widths 6
6.3. Settings and styles 6
7. Queues and stacks 7
7.1. \hStack: a stack in an array 7
7.1.1. Step by step 7
7.2. \hQueue: a circular queue in an array 7
7.2.1. Empty, first index 0, bigger boxes 7
7.2.2. Step by step 8
8. Trees 8
8.1. A tree with TikZ's child syntax 8
8.2. An expression tree: hsquare 8
9. Git commit graphs 9
10. In a slide deck 9
11. Customizing 9
12. Troubleshooting 10
References 10

1. Getting started

```latex
\documentclass[11pt]{amsart}
\usepackage{../../cis000_latex/lib/hblibArticle}          % an article; or hblibPresentation in a deck
\usepackage{../../cis000_latex/lib/hblibDatastructures}   % after it
...
\begin{figure}[h]
  \centering
  \begin{tikzpicture}
    \hArray[name=a]{3,1,4,1,5}
  \end{tikzpicture}
  \caption{An array.}\label{fig:array}
\end{figure}
```

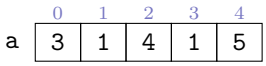


FIGURE 1. An array.

- Every macro is used *inside* a `tikzpicture`; the picture can have more nodes and arrows. Put it in a `figure` to give it a caption and a label, then refer to it with `\reffig{fig:array}` (Fig. 1).
- Lengths are in cm.
- Load it after hblibArticle (articles) or hblibPresentation (slide decks). It needs only xcolor, tikz and fp, no other hb library.
- It loads the TikZ libraries positioning (`right=of a`), arrows.meta, decorations.pathreplacing and shapes.multipart (the nodes of linked lists).

Draws	**With**
queue, stack	`\hQueue`, `\hStack` (array implementations as in CLRS [**?**]), Sec. 7
string, array	`\hString`, `\hArray`; styles `hvar`, `href` for references, Sec. 4.2
two-dimensional array	`\hGrid`, Sec. 5
linked lists	`\hDLList` (doubly linked), `\hSLList` (singly linked), Sec. 6
trees	TikZ styles `htree`, `hnode`, `hsquare`, ..., Sec. 8
commit graphs	TikZ styles `hcommit`, `hbranch`, ..., Sec. 9

2. Quick reference

Every command and style, with the section that shows examples. Optional arguments are in `[brackets]`.

Write	**Draws**
`\hQueue{first}{capacity}{head}{tail}{e1,...}[w][h]`	a circular queue, Sec. 7.2
`\hStack{first}{capacity}{top}{e1,...}[w][h]`	a stack, Sec. 7.1
`\hString[w]{c,h,a,r,s}[from][to]`	a string, a substring highlighted, Sec. 4.1
`\hArray[keys]{e0,e1,...}`	an array; nodes `(id-i)`, Sec. 4.2
`\hGrid[keys]{{row 0},{row 1},...}`	a 2D array; nodes `(id-r-c)`, Sec. 5
`\hDLList{name}{x}{y}{v1,...}{new}`	a doubly linked list, Sec. 6.1
`\hSLList{name}{x}{y}{v1,...}{new}`	a singly linked list, Sec. 6.2
`hvar`, `href`	a reference variable and its arrow, Sec. 4.2.2
`htree`, `hnode`, `hsquare`, `hhot`, `hdone`, `hleaf`, `horder=k`	trees, Sec. 8
`hcommit`, `hbranch`, `hheadref`, `htag`, `hparent`	commit graphs, Sec. 9

3. Colors

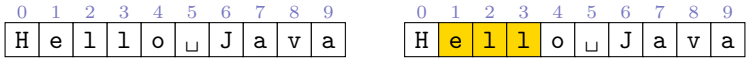
Name	**RGB**	**Used for**
colorTop	55, 215, 0	top of a stack; the current cell (`cur=`); `hhot`
colorTail	55, 215, 0	tail of a queue
colorHead	255, 215, 0	head of a queue; highlighted cells (`hot=`)
colorEmpty	170, 255, 0	an empty queue (head = tail)
colorUsed	238, 107, 110	cells in use; `colorUsed!55`: visited (`done=`)
colorUnused	230, 230, 230	cells not in use (`unused=`)
colorLabel	250, 118, 194	the h and t marks of a queue
colorIndex	119, 118, 194	index numbers; `href` arrows
colorBoxZero	160, 160, 160	the unused cell 0 when indices start at 1

Change one with `\definecolor` after loading, e.g. `\definecolor{colorHead}{RGB}{255,180,0}`. colorIndex here (purple) replaces the blue colorIndex of hblibCommon. The linked lists use their own TikZ styles (Sec. 6.3).

4. Strings and arrays

4.1. \hString: characters and substring

```latex
\hString[box width]{c,h,a,r,s}[from][to]        % box width 0.45 cm; boxes from <= i < to highlighted
\hString{H,e,l,l,o,\textvisiblespace,J,a,v,a}
\hString{H,e,l,l,o,\textvisiblespace,J,a,v,a}[1][4]    % s.substring(1, 4) is "ell"
```



- One box per character, the 0-based index above it.
- \textvisiblespace shows a space (␣); a comma must be given in braces: {,}.
- The highlighted boxes are exactly the characters of s.substring(from, to).

4.2. \hArray: the keys

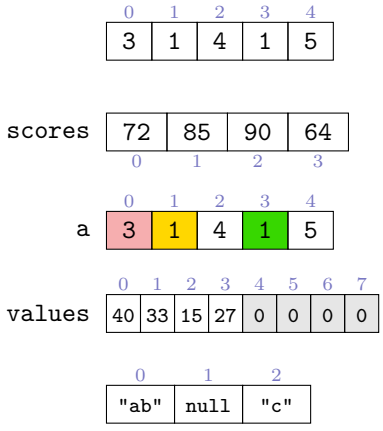
\hArray[keys]{e0,e1,...}: one box per element with its 0-based index.

Key	Default	Meaning
id=a	hA	cell i is the node (a-i), e.g. (a-0.west)
at={(x,y)}	(0,0)	lower-left corner of cell 0
width=, height=	0.6, 0.5	box size in cm
name=scores	none	a label on the left
index=	above	above, below or none
hot={2}	none	highlighted cells (colorHead)
cur={3}	none	the current cell (colorTop)
done={0,1}	none	visited cells (colorUsed!55)
unused={5,...,7}	none	cells not in use (colorUnused)
font=	\ttfamily\small	font of the elements

Lists use \foreach syntax: hot={0,...,3}. An element with a comma goes in braces: {{1,2},x}.

4.2.1. Keys in use

```latex
\hArray[at={(1.2,4.4)}]{3,1,4,1,5}
\hArray[at={(1.2,3.2)}, name=scores, index=below, width=0.8]{72,85,90,64}
\hArray[at={(1.2,1.9)}, name=a, hot={1}, cur={3}, done={0}]{3,1,4,1,5}
\hArray[at={(1.2,0.8)}, name=values, unused={4,...,7}, width=0.45, font=\ttfamily\scriptsize]{40,33,15,27,0,0,0,0}
\hArray[at={(1.2,-0.4)}, font=\ttfamily\scriptsize, width=0.9]{"ab",null,"c"}
```

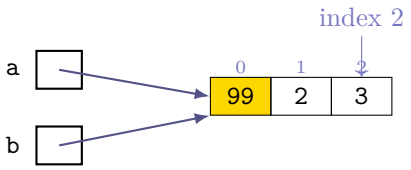


Several arrays in one picture: give each its own at={(x,y)} (here they are stacked).

4.2.2. Node names, labels and reference arrows

```latex
\hArray[id=r, at={(2.6,0)}, width=0.8, hot={0}]{99,2,3}
\node[hvar] (a) at (0.6,0.6) {};
\node[font=\ttfamily\small, left=2pt of a] {a};
\draw[href] (a.center) -- (r-0.west);
\node[hvar] (b) at (0.6,-0.4) {};
\node[font=\ttfamily\small, left=2pt of b] {b};
```

```
\draw[href] (b.center) -- (r-0.south west);
\node[colorIndex, font=\small] (ix) at (4.6,1.3) {index 2};
\draw[->, colorIndex] (ix) -- (r-2.north);
```



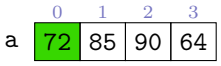
- `hvar`: a variable box; `href`: the arrow of a reference.
- Two arrows to one array: `a` and `b` are aliases.
- `left=2pt of a` uses TikZ's positioning library (loaded by hblibDatastructures).

4.2.3. A loop, step by step

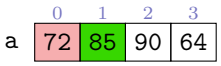
`cur` moves, `done` grows; the label follows `i`; at `i = 4` the loop stops.

latex
```
\hArray[name=a, cur={0}]{72,85,90,64}                 % i = 0
\hArray[name=a, done={0}, cur={1}]{72,85,90,64}       % i = 1
\hArray[name=a, done={0,1}, cur={2}]{72,85,90,64}     % i = 2
\hArray[name=a, done={0,...,2}, cur={3}]{72,85,90,64}   % i = 3
\hArray[name=a, done={0,...,3}]{72,85,90,64}          % i = 4: the loop stops
```

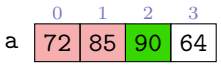
`i = 0`



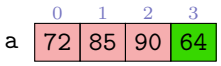
`i = 1`



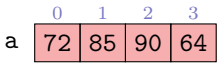
`i = 2`



`i = 3`



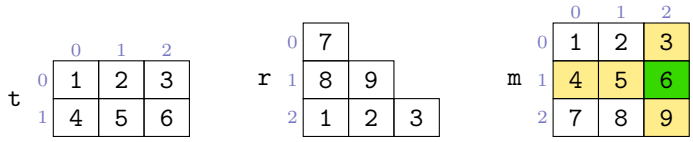
`i = 4`: stop



5. Two-dimensional arrays

`\hGrid[keys]{{row 0},{row 1},...}`: one brace group per row, row 0 on top.

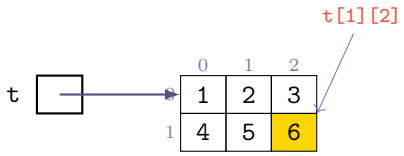
latex
```
\hGrid[name=t]{{1,2,3},{4,5,6}}                       % a 2 x 3 table
\hGrid[name=r, index=rows]{{7},{8,9},{1,2,3}}         % ragged: rows of different lengths
\hGrid[name=m, rowhot={1}, colhot={2}, cur={1/2}]{{1,2,3},{4,5,6},{7,8,9}}
```



- Cell `(r,c)` is the node `(<id>-r-c)`, e.g. `(g-1-2.east)`.
- Keys as in `\hArray` (`id`, `at`, `width`, `height`, `name`, `font`); cells are given as `r/c`: `hot={1/2,0/0}`, `cur`, `done`, `unused`.
- `rowhot={1}`, `colhot={2}`: whole rows, whole columns; `index= both` (default), `rows`, `cols` or `none`.

5.1. An array of rows

latex
```
\hGrid[id=g, at={(1.6,0)}, hot={1/2}]{{1,2,3},{4,5,6}}
\node[hvar] (t) at (0,0.25) {};
\node[font=\ttfamily\small, left=2pt of t] {t};
\draw[href] (t.center) -- (g-0-0.west);
\node[font=\small, colorIndex] (c) at (4.0,1.3) {\hCode{t[1][2]}};
\draw[->, colorIndex] (c) -- (g-1-2.north east);
```



Row r is drawn $r\times$ height lower than row 0; the cell nodes make arrows and labels easy.

6. Linked lists

```latex
\hDLList{name}{x}{y}{v1,v2,...}{new}   % a doubly linked list
\hSLList{name}{x}{y}{v1,v2,...}{new}   % a singly linked list
```

Argument	Meaning
`name`	the list variable, printed in its box, e.g. `list`; also the prefix of the node names
`x`, `y`	the variable box is centred at (`x`, `y` + `\dlVarRaise`); the nodes are drawn at height `y`
`v1,v2,...`	the values, comma separated; empty: the empty list, the variable points to `null`
`new`	the index (1, 2, ...) of the node highlighted in orange, e.g. a node just added; 0: none

- The variable's link leaves its box to the right and meets the first node vertically, at its upper-left corner.
- `next` links (blue) curve above the nodes; `prev` links (red) curve below them.
- An end of the list points to `null`, written where one more node would be.

6.1. `\hDLList`: a doubly linked list

A node is `prev | data | next`: `prev` and `next` are narrow, `data` fits the value. The first node's `prev` and the last node's `next` point to `null`.

```latex
\begin{figure}[h]
  \centering
  \begin{tikzpicture}
    \hDLList{list}{0}{0}{1,5,4}{0}
  \end{tikzpicture}
  \caption{list -> [1, 5, 4]}\label{fig:dll}
\end{figure}
```

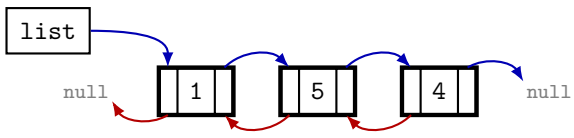
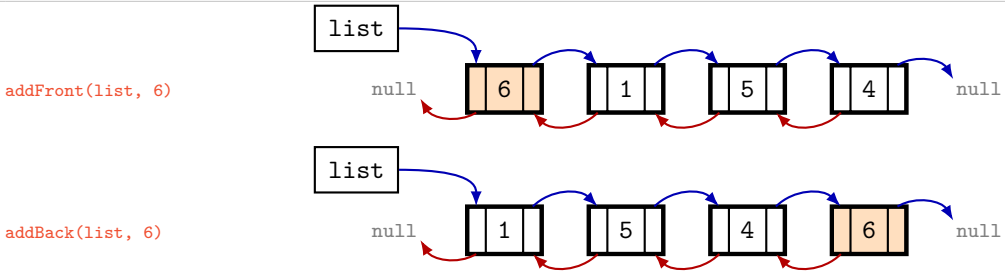


FIGURE 2. `list -> [1, 5, 4]`. Each node is `prev | data | next`; blue: `next` links, red: `prev` links.

6.1.1. After adding a node

The last argument highlights the new node.

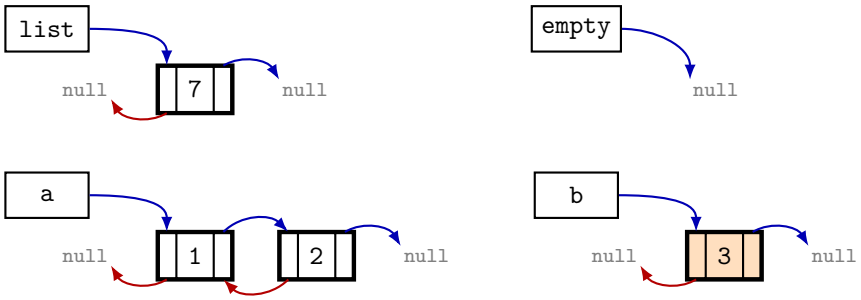
```latex
\hDLList{list}{0}{0}{6,1,5,4}{1}   % after list = addFront(list, 6): node 1 is new
\hDLList{list}{0}{0}{1,5,4,6}{4}   % after list = addBack(list, 6):  node 4 is new
```



6.1.2. One node, the empty list, two lists in one picture

Each list gets its own place with `x` and `y`, and its own `name`.

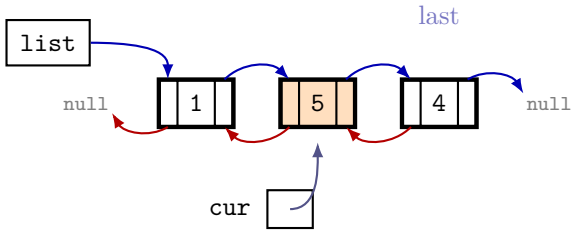
```latex
\hDLList{list}{0}{0}{7}{0}       % one node: both of its links are null
\hDLList{empty}{7}{0}{}{0}       % the empty list: empty -> null
\hDLList{a}{0}{-2.2}{1,2}{0}     % a second row, 2.2 cm lower
\hDLList{b}{7}{-2.2}{3}{1}
```



6.1.3. Node names: labels and arrows of your own

With `name` = `list` the nodes are `(list-var)`, `(list-1)`, `(list-2)`, ..., `(list-nullprev)` and `(list-nullnext)`. Any anchor works, e.g. `(list-2.north)`.

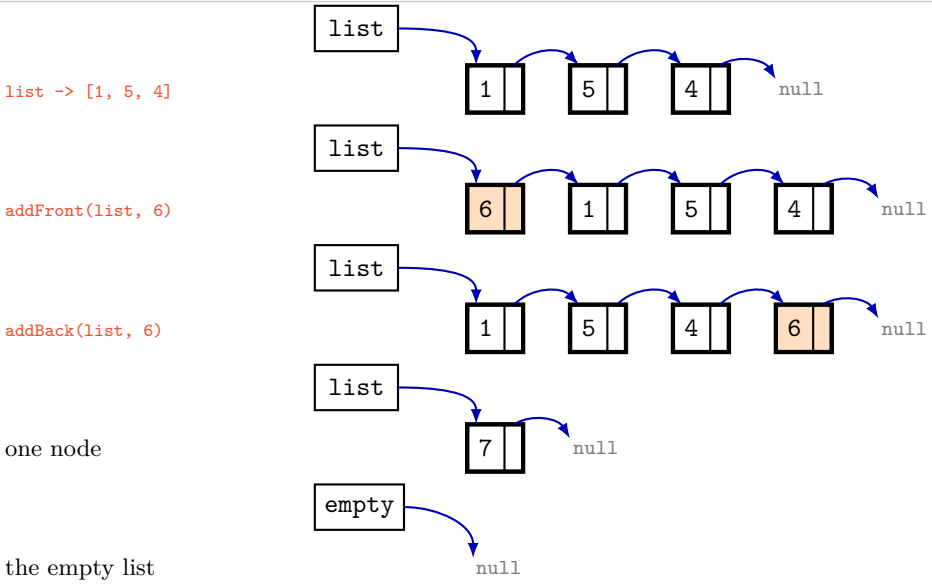
```latex
\hDLList{list}{0}{0}{1,5,4}{2}
\node[hvar] (cur) at (3.2,-1.4) {};
\node[font=\ttfamily\small, left=2pt of cur] {cur};
\draw[href] (cur.center) to[out=0, in=-90] ([yshift=-2mm]list-2.south);
\node[font=\small, colorIndex, above=6mm of list-3] {last};
```



6.2. `\hSLList`: a singly linked list

The same arguments as `\hDLList`. A node is `data | next`; there are no `prev` links and no `null` on the left.

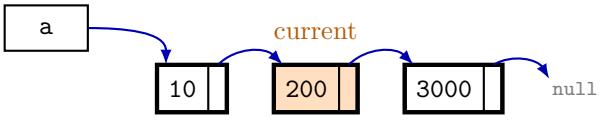
```latex
\hSLList{list}{0}{0}{1,5,4}{0}       % list -> [1, 5, 4]
\hSLList{list}{0}{0}{6,1,5,4}{1}     % after list = addFront(list, 6)
\hSLList{list}{0}{0}{1,5,4,6}{4}     % after list = addBack(list, 6)
\hSLList{list}{0}{0}{7}{0}           % one node
\hSLList{empty}{0}{0}{}{0}           % the empty list
```



6.2.1. Values of different widths

The `data` part fits its value; the gaps between the nodes stay the same. Node names as in Sec. 6.1.3, without `(name-nullprev)`.

```latex
\hSLList{a}{0}{0}{10,200,3000}{2}
\node[font=\small, text=orange!70!black, above=2mm of a-2] {current};
```



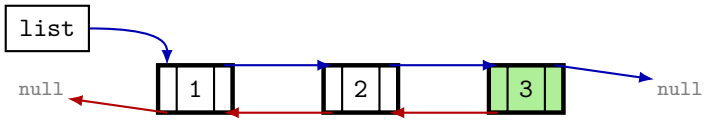
6.3. Settings and styles

Both macros share these settings. Change one with `\renewcommand` after loading; inside a group `{...}` it changes only that part.

Setting	Default	Meaning
`\dlVarRaise`	0.8	how far (cm) the variable box sits above the nodes
`\dlFirstGap`	0.9cm	from the variable box to the first node
`\dlGap`	0.62cm	between two nodes, and between an end node and `null`
`\dlBend`	45	how much the links curve (degrees; 0: straight)
`\dlTip`	3.4pt	where a link starts and ends: this far in from a node's corner

Style	Used for
`dlnode`, `dlnew`	a node of `\hDLList`; the highlighted one (orange)
`slnode`, `slnew`	a node of `\hSLList`; the highlighted one
`dlvar`	the variable box
`dlnext`, `dlprev`	`next` links (and the variable's link), `prev` links
`dlnull`	the word `null`
`dlframe`	the thick outer frame of a node (1.6 pt)

```latex
{\renewcommand{\dlGap}{1.2cm}\renewcommand{\dlBend}{0}          % this picture only
 \tikzset{dlnew/.style={dlnode, fill=colorTop!40}}               % a green new node
 \begin{tikzpicture} \hDLList{list}{0}{0}{1,2,3}{3} \end{tikzpicture}}
```



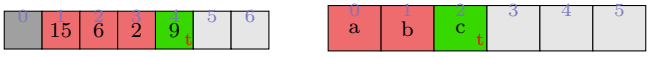
7. Queues and stacks

7.1. `\hStack`: a stack in an array

```latex
\hStack{first}{capacity}{top}{e1,e2,...}[box width][box height]   % widths in cm: 0.5, 0.5
```

- `first` as in `\hQueue`; `top`: the index of the top element (green, mark t).
- Below top: in use (red); above top: free (gray). Top below the first index: an empty stack.

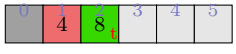
```latex
\hStack{1}{6}{4}{15,6,2,9}
\hStack{0}{6}{2}{a,b,c}[0.7][0.6]
```



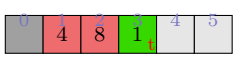
7.1.1. Step by step

```latex
\hStack{1}{5}{2}{4,8}        % a stack 4, 8
\hStack{1}{5}{3}{4,8,1}      % push 1
\hStack{1}{5}{2}{4,8}        % pop: 1 leaves
```

a stack 4, 8



push 1



pop: 1 leaves



7.2. \hQueue: a circular queue in an array

```
\hQueue{first}{capacity}{head}{tail}{e1,e2,...}[box width][box height]   % widths in cm: 0.8, 0.6
```
latex

- `first`: 1 = indices 1 … capacity, cell 0 gray and not used (CLRS); 0 = indices 0 … capacity−1.
- `head`: the first element (yellow, mark h); `tail`: the next free cell (green, mark t).
- In use: from head to tail−1, wrapping around (red); head = tail: empty (light green).
- The list fills the cells from the first index on; leave a place empty with nothing between commas.

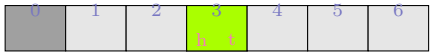
```
\hQueue{1}{6}{2}{5}{,15,6,9}       % the queue 15, 6, 9; index 1 empty
\hQueue{1}{6}{5}{2}{3,,,,8,4}      % wrapped around: the queue 8, 4, 3
```
latex





7.2.1. Empty, first index 0, bigger boxes

```
\hQueue{1}{6}{3}{3}{}              % empty
\hQueue{0}{6}{1}{4}{,5,7,2}        % indices 0..5
\hQueue{0}{4}{0}{2}{a,b}[1.0][0.7] % bigger boxes
```
latex







7.2.2. Step by step

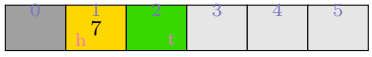
In an article each step is its own picture; in a slide deck use one `\only<n>` per step (Sec. 10).

```
\hQueue{1}{5}{1}{1}{}        % empty
\hQueue{1}{5}{1}{2}{7}       % enqueue 7
\hQueue{1}{5}{1}{3}{7,2}     % enqueue 2
\hQueue{1}{5}{2}{3}{,2}      % dequeue: 7 leaves
```
latex

empty



enqueue 7



enqueue 2



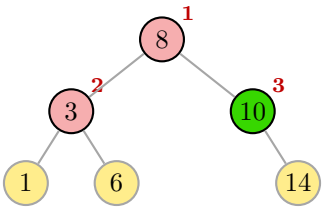
dequeue: 7 leaves



8. Trees

8.1. A tree with TikZ's child syntax

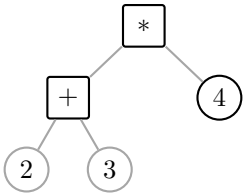
```
\begin{tikzpicture}[htree,
    level 1/.style={sibling distance=2.4cm},
    level 2/.style={sibling distance=1.2cm}]
  \node[hnode, hdone, horder=1] {8}
    child {node[hnode, hdone, horder=2] {3}
      child {node[hnode, hleaf] {1}}
      child {node[hnode, hleaf] {6}}}
    child {node[hnode, hhot, horder=3] {10}
      child[missing]
      child {node[hnode, hleaf] {14}}};
\end{tikzpicture}
```
latex



- `htree`: level distance, gray edges; `hnode`: a round node.
- `hhot`: current; `hdone`: visited; `hleaf`: a leaf.
- `horder=k`: the visit number k; `child[missing]`: an empty place.

8.2. An expression tree: `hsquare`

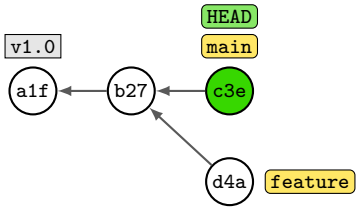
```latex
\begin{tikzpicture}[htree, level 1/.style={sibling distance=2cm},
    level 2/.style={sibling distance=1.1cm}]
  \node[hsquare] {$*$}
    child {node[hsquare] {$+$}
      child {node[hnode] {2}}
      child {node[hnode] {3}}}
    child {node[hnode] {4}};
\end{tikzpicture}
```



$(2+3)*4$: operators in squares, numbers in circles. `hsquare` also suits folders.

9. Git commit graphs

```latex
\node[hcommit] (c1) at (0,0)     {a1f};
\node[hcommit] (c2) at (1.3,0)   {b27};
\node[hcommit, hhot] (c3) at (2.6,0) {c3e};
\node[hcommit] (f1) at (2.6,-1.2) {d4a};
\draw[hparent] (c2) -- (c1);
\draw[hparent] (c3) -- (c2);
\draw[hparent] (f1) -- (c2);
\node[hbranch, above=3pt of c3] (m) {main};
\node[hheadref, above=2pt of m] {HEAD};
\node[hbranch, right=4pt of f1] {feature};
\node[htag, above=3pt of c1] {v1.0};
```



- An arrow points from a commit to its *parent*.
- `hbranch`: a branch name; `hheadref`: HEAD; `htag`: a tag.
- `hhot`, `hdone` color a commit, as in trees.

10. In a slide deck

The macros work the same in beamer. Load `hblibPresentation` instead of `hblibArticle`. A frame with code listings must be `[fragile]`. Steps go on the slides of one frame with `\only<n>`: one picture per slide.

```latex
\documentclass[aspectratio=169,10pt,compress]{beamer}
\usepackage{../../hblib/hblibPresentation}
\usepackage{../../hblib/hblibDatastructures}
```

```
...
\begin{frame}[t] \frametitle{addFront}
  \begin{center}
    \begin{tikzpicture}
      \only<1>{\hDLList{list}{0}{0}{1,5,4}{0}}
      \only<2>{\hDLList{list}{0}{0}{6,1,5,4}{1}}
    \end{tikzpicture}
  \end{center}
\end{frame}
```

Note. Overlays (`\only<2>`) are for slides only; in an article draw each step in its own picture, as in this manual.

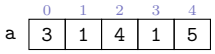
11. Customizing

Put these after `\usepackage{hblibDatastructures}`; inside a group `{...}` they change only that part.

```latex
\definecolor{colorHead}{RGB}{255,180,0}          % a darker yellow for head and hot cells
\definecolor{colorIndex}{RGB}{39,138,194}        % blue index numbers
\tikzset{hnode/.append style={minimum size=0.7cm}}         % bigger tree nodes
\renewcommand{\dlGap}{0.8cm}                     % more space between list nodes
\tikzset{dlnext/.append style={green!50!black}} % green next links
```

One picture only: give the TikZ options in brackets or use a group.

```latex
\begin{tikzpicture}[scale=0.8, transform shape]   % 80 % of the size, text too
  \hArray[name=a]{3,1,4,1,5}
\end{tikzpicture}
```



12. Troubleshooting

Message or problem	Cause	Fix
Missing \begin{document}, or nodes in odd places	a macro outside a `tikzpicture`	put it inside one
two arrays (or lists) drawn on top of each other	both at the same place	`at={(x,y)}`; for lists `x`, `y`
"No shape named a-3"	wrong `id` or `name`, or a cell or node that does not exist	arrays: cells $0 \ldots n-1$; lists: nodes $1 \ldots n$
an element splits in two	a comma in the element	braces: `{1,2}`
`\hQueue` elements in the wrong cells	the list starts at the first index	leave cells empty: `{,15,6}`
"I do not know the key ..."	a misspelled key of `\hArray` or `\hGrid`	see the keys table
"... of a" does not work	a library older than v20261108T0900	update hblibDatastructures.sty (it loads positioning)
"Undefined control sequence \hDLList"	a library older than v20261112T0930	update hblibDatastructures.sty
a list shows only `null`	the values are empty: the empty list	give the values: `{1,5,4}`
two lists in one picture share node names	the same `name`	a different `name` for each list
a setting changed every later picture	`\renewcommand` outside a group	put it in `{...}`
indices purple, not blue	this library's colorIndex	`\definecolor{colorIndex}{...}`

References